

Collaborative Development of a Team Portfolio Website

Deadline : Oct 23, 2025

1. Preamble

This assignment gives you a hands-on experience with distributed version control in a setting similar to real professional software development. The main goal is to help you go beyond theory and actually use Git and GitHub in a real teamwork situation.

You will work together as a development team, following industry-standard workflows to create a shared software project. The activity is meant to be challenging, showing what it's like to work on a project with multiple developers.

The focus is not only on the final product, but also on **how you collaborate and follow the workflow**. In other words, the process of working together is just as important as the final result.

2. Objectives

Upon completing this assignment, each student will be able to:

- ❖ Use the feature-branching Git workflow to work on different parts of a project at the same time.
- ❖ Review teammates' code through pull requests and give helpful technical feedback.
- ❖ Solve merge conflicts carefully when combining different code changes in the same project.
- ❖ Collaborate effectively by managing shared code in a remote GitHub repository.
- ❖ Publish a static website using GitHub Pages as a hosting platform.

3. Project Specification

The team will engineer and deploy a static "Team Portfolio" website. The system must be hosted on GitHub Pages and meet the following functional requirements.

3.1 System Components

- index.html (Root Document): A primary entry point that enumerates all team members. Each member's name must function as a hyperlink navigating to their corresponding individual profile page.
- Individual Profile Documents (firstname-lastname.html): Each team member is responsible for the creation of a unique HTML document. This document must contain:
 - The full name of the student.
 - A concise professional biography (2-3 sentences).
 - An itemized list of technical competencies (e.g., Python, Java, CSS).
 - A hyperlink to their personal GitHub profile.

3.2 Prerequisites and Environment Configuration

- ❖ The team leader must be designated as the Repository Administrator.
- ❖ All team members must possess active GitHub accounts and have Git VCS installed locally.
- ❖ Local Git environments must be configured with the user's full name and email address.
- ❖ A synchronous communication channel (e.g., Slack, Discord) must be established for the team.

4. Methodology and Procedural Guidelines

Adherence to the following phased methodology is mandatory.

Phase 1: Repository Initialization and Configuration

Responsible: Repository Administrator (Team Lead)

1.1 Repository Instantiation: A new, public GitHub repository shall be created with the name team-portfolio-project. The repository must be initialized with a README.md file.

1.2 Initial Commit: The README.md file will be modified in the initial commit to enumerate all team members and their GitHub usernames.

1.3 Collaborator Access: All team members shall be invited to the repository as collaborators via their GitHub usernames. The administrator must verify that all invitations have been accepted before proceeding.

1.4 Branch Creation: A develop branch shall be created from the main branch and pushed to the remote repository. This branch will serve as the primary integration point for all feature development.

```
git checkout -b develop
```

```
git push -u origin develop
```

1.5 Implementation of Branch Protection Rules: A branch protection rule must be applied to the main branch to enforce quality gates. The rule must mandate:

- All merges must be enacted via a pull request.
- A minimum of one approving review is required before merging.

Phase 2: Distributed Development and Integration

Responsible: All Team Members

2.1 Repository Cloning: Each team member shall clone the remote repository to their local machine.

2.2 Feature Branch Creation: Prior to commencing work, each member must synchronize their local `develop` branch with the remote repository. Subsequently, a descriptively named feature branch (e.g., `feature/implement-firstname-lastname-profile`) must be created from the `develop` branch.

```
git checkout develop
```

```
git pull origin develop
```

```
git checkout -b <your-feature-branch-name>
```

2.3 Task Implementation: Within the feature branch, each member will create their individual HTML profile and modify the `index.html` document to include a hyperlink to their new page.

2.4 Atomic Commits and Conventional Commit Standards : Changes must be committed in logical, atomic units. Each commit should represent a single, coherent change to the codebase. Commit messages must adhere to the **Conventional Commits** standard, clearly articulating the purpose and scope of the change.

Conventional Commit Format

All commit messages must follow this structure:

```
<type>: <description>
```

Where:

- **type:** Categorizes the nature of the change (see table below)
- **description:** A concise summary in imperative mood (50 characters or less)

Commit Type Reference

Type	When to Use	Example for This Project
feat:	Adding new functionality or features	feat: Create profile page for First Last feat: Add navigation menu to index page
fix:	Fixing errors, bugs, or conflicts	fix: Resolve merge conflict in team list fix: Correct broken GitHub profile link
refactor:	Improving or restructuring existing code without changing behavior	refactor: Reorganize CSS classes for better readability refactor: Improve HTML semantic structure
docs:	Documentation changes only	docs: Add project description to README docs: Update team member list in README
style:	Code formatting, whitespace, indentation (no logic changes)	style: Fix indentation in HTML files style: Add consistent spacing in CSS
chore:	Maintenance tasks, configuration updates	chore: Update .gitignore file chore: Configure GitHub Pages settings

Commit Message Best Practices

- Use **lowercase** for the type prefix
- Use **imperative mood**: "Add feature" not "Added feature" or "Adding feature"
- Keep descriptions **concise and specific** (aim for 50 characters or less)
- **Do not** end with a period
- Make each commit **atomic**: one logical change per commit

Commit Message Best Practices

- Use lowercase for the type prefix
- Use imperative mood: "Add feature" not "Added feature" or "Adding feature"
- Keep descriptions concise and specific (aim for 50 characters or less)
- Do not end with a period
- Make each commit atomic: one logical change per commit

Example Commits

```
git commit -m "feat: Implement structure for Firstname Lastname profile page"
```

```
git commit -m "refactor: Add Firstname Lastname hyperlink to index document"
```

2.5 Push to Remote: The local feature branch must be pushed to the remote repository to make it visible to the team.

```
git push -u origin <your-feature-branch-name>
```

2.6 Pull Request Submission: A pull request shall be opened on GitHub to merge the feature branch into the `develop` branch. The pull request description must provide a concise summary of the implemented changes.

2.7 Peer Code Review: Each team member is required to formally review a minimum of two pull requests from other members. Reviews should focus on code quality, correctness, and adherence to project specifications. Approval or requests for changes should be submitted through the GitHub interface.

2.8 Conflict Resolution and Branch Synchronization: As pull requests are merged into `develop`, feature branches will become desynchronized, leading to merge conflicts, particularly within `index.html`. It is the responsibility of each branch owner to resolve these conflicts by merging the latest `develop` branch into their feature branch, manually resolving the code conflicts, committing the resolution, and pushing the fix.

From your feature branch:

```
git pull origin develop
```

Git will report a conflict. Open the conflicted file(s).

Manually edit the file(s) to resolve discrepancies and remove

conflict markers.

```
git add .
```

```
git commit -m "fix: Resolve merge conflict with upstream develop"
```

```
git push
```

2.9 Pull Request Merging: Once a pull request has received the required approval and is free of conflicts, it may be merged into `develop`. The feature branch should be deleted post-merge to maintain repository hygiene.

Phase 3: Final Integration and Deployment

Responsible: Repository Administrator (Team Lead)

3.1 Final Integration: After all feature branches have been successfully merged into develop, the Repository Administrator will initiate a final pull request to merge the develop branch into main. This PR requires at least one final approval.

3.2 Deployment: The project will be deployed using GitHub Pages. The deployment source must be configured to the main branch.

3.3 Project Documentation and Retrospective Analysis: The README.md file must be updated with the final submission deliverables.

5. Submission Deliverables

The final submission consists of a single URL to the public GitHub repository. The README.md file within this repository must contain:

- The definitive list of all contributing team members.
- A hyperlink to the deployed website on GitHub Pages.
- A Team Retrospective Analysis (approximately 250-400 words): A concise technical report addressing the following prompts:
 - Identify and analyze the most significant technical challenge encountered during the collaborative workflow.
 - Provide a detailed description of a specific merge conflict, outlining the procedural steps taken to achieve resolution.
 - Evaluate the efficacy of the pull request and peer review process in enhancing the final quality of the software artifact.

Good luck !